

Отчёт по лабораторной работе №12

Дисциплина: Операционные системы

Терентеевская Светлана Александровна

Содержание

1	Цель работы	5
2	Задание	6
3	Теоретическое введение	7
3.1	Стандартные переменные	7
3.2	Метасимволы и экранирование Метасимволы — символы, имеющие специальный смысл: * ? [] > < & \ \$; # ` () { } ' "	8
3.3	Командные файлы	8
3.4	Функции	8
3.5	Специальные переменные для командных файлов	9
3.6	Команда getopt	9
3.7	Управляющие конструкции	10
4	Выполнение лабораторной работы	12
5	Выводы	18
6	Список литературы	19

Список иллюстраций

4.1	Открытие файла	12
4.2	Скрипт для backup_self.sh	12
4.3	Запуск backup_self.sh	12
4.4	Проверка архива	13
4.5	Скрипт для args.sh	13
4.6	Запуск args.sh	14
4.7	Скрипт для myls.sh	15
4.8	Скрипт для myls.sh	15
4.9	Запуск myls.sh	16
4.10	Скрипт для format.sh	17
4.11	Количество txt-файлов	17
4.12	Количество sh-файлов	17

Список таблиц

1 Цель работы

Изучить основы программирования в оболочке ОС UNIX/Linux. Научиться писать небольшие командные файлы.

2 Задание

- Сделать резервную копию самого скрипта в каталог ~/backup с использованием архиватора.
- Обработать произвольное число аргументов командной строки.
- Вывести содержимое каталога и права доступа к файлам без использования ls и dir.
- Подсчитать количество файлов с заданным расширением в указанной директории.

3 Теоретическое введение

3.1 Стандартные переменные

Переменная	Назначение
PATH	Список каталогов для поиска команд (разделитель — двоеточие). По умолчанию: текущий каталог, /bin, /usr/bin
HOME	Домашний каталог пользователя
IFS	Разделители полей в командной строке (пробел, табуляция, перевод строки)
MAIL	Файл для проверки почты
TERM	Тип используемого терминала
LOGNAME	Регистрационное имя пользователя

Просмотреть значения всех переменных можно командой `set`.

3.2 Метасимволы и экранирование

Метасимволы — символы, имеющие специальный смысл: * ? [] > < | & \ \$; # ` () { } ' " .

Назначение основных метасимволов:

- * — любая строка (в том числе пустая)
- ? — один любой символ
- [c1-c2] — любой символ из диапазона

Экранирование — снятие специального смысла:

- Обратный слеш перед метасимволом
- Одинарные кавычки (полное экранирование)
- Двойные кавычки (частичное экранирование)

3.3 Командные файлы

Командный файл (скрипт) — текстовый файл с последовательностью команд. Для запуска необходимо установить право на выполнение и указать путь к файлу.

3.4 Функции

Функция определяется один раз в начале скрипта и может вызываться по имени.

3.5 Специальные переменные для командных файлов

Переменная	Назначение
\$0	Имя скрипта
\$1...\$9	Аргументы командной строки (для 10-го и более — \${10})
\$#	Количество аргументов
\$@	Все аргументы как отдельные слова
\$*	Все аргументы как одна строка
\$?	Код завершения последней выполненной команды (0 — истина, ненулевое — ложь)
\$\$	PID текущего процесса
\$_	PID последнего фонового процесса
\${#name}	Длина строки в переменной name
\${name:-value}	Если переменная не определена, подставляется значение
\${name:=value}	Если переменная не определена, ей присваивается значение

3.6 Команда getopt

`getopts` осуществляет синтаксический анализ командной строки, выделяя флаги (опции). Строка опций содержит список допустимых флагов; если после флага ожидается аргумент, ставится двоеточие. `getopts` использует переменные `OPTARG` (аргумент флага) и `OPTIND` (индекс следующего аргумента). Обычно `getopts` применяется в цикле `while` совместно с оператором `case`.

3.7 Управляющие конструкции

3.7.1 Цикл `for`

Перебирает элементы списка, последовательно присваивая их переменной и выполняя заданные команды.

3.7.2 Оператор выбора `case`

Сравнивает значение переменной с несколькими шаблонами и выполняет команды, соответствующие первому совпавшему шаблону. Метасимвол `*` используется как шаблон по умолчанию.

3.7.3 Условный оператор `if`

Выполняет проверку условия. Если последняя команда в списке после `if` возвращает нулевой код (истина), выполняется блок `then`. Если нет — проверяется `elif` или выполняется блок `else`. Завершается конструкция служебным словом `fi`.

Команда `test` используется для проверки условий:

- `-d` — проверка, является ли файл каталогом
- `-f` — проверка, является ли файл обычным файлом
- `-r` — проверка доступа на чтение
- `-w` — проверка доступа на запись
- `-x` — проверка доступа на выполнение

3.7.4 Цикл `while`

Выполняется, пока последняя команда в списке условий возвращает нулевой код (истина).

3.7.5 Цикл `until`

Выполняется, пока последняя команда в списке условий возвращает ненулевой код (ложь). Условие выхода из цикла —противоположное `while`.

3.7.6 Команды `true` и `false`

- `true` — всегда возвращает нулевой код (истина)
- `false` — всегда возвращает ненулевой код (ложь)

3.7.7 Прерывание циклов

- `break` — полностью завершает выполнение цикла
- `continue` — завершает текущую итерацию и переходит к следующей

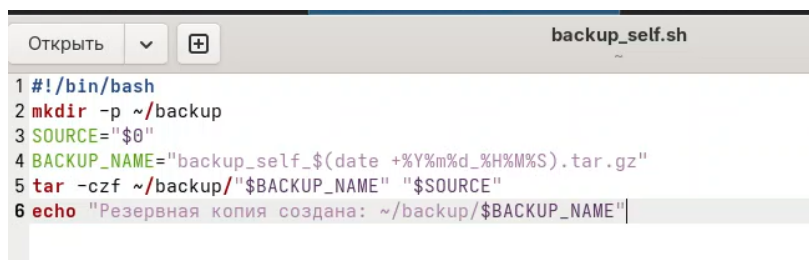
4 Выполнение лабораторной работы

1. Я создала и открыла файл backup_self.sh (рис. 4.1).

```
saterenteevskaya@saterenteevskaya:~$ cd  
saterenteevskaya@saterenteevskaya:~$ gedit backup_self.sh
```

Рисунок 4.1: Открывание файла

Написала в нем скрипт, который при запуске будет делала резервную копию самого себя (то есть файла, в котором содержится его исходный код) в другую директорию backup в моём домашнем каталоге (рис. 4.2).



```
1 #!/bin/bash  
2 mkdir -p ~/backup  
3 SOURCE="$0"  
4 BACKUP_NAME="backup_self_$(date +%Y%m%d_%H%M%S).tar.gz"  
5 tar -czf ~/backup/"$BACKUP_NAME" "$SOURCE"  
6 echo "Резервная копия создана: ~/backup/$BACKUP_NAME" |
```

Рисунок 4.2: Скрипт для backup_self.sh

- Запустила (рис. 4.3).

```
saterenteevskaya@saterenteevskaya:~$ chmod +x backup_self.sh  
saterenteevskaya@saterenteevskaya:~$ ./backup_self.sh  
Резервная копия создана: ~/backup/backup_self_20260407_175633.tar.gz  
saterenteevskaya@saterenteevskaya:~$ |
```

Рисунок 4.3: Запуск backup_self.sh

- Проверила, что создался архив (2 раза) (рис. 4.4).

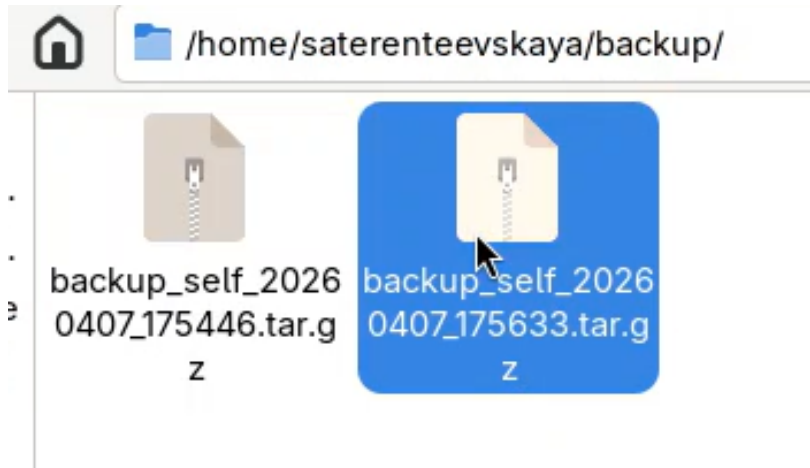


Рисунок 4.4: Проверка архива

2. Я написала пример командного файла, обрабатывающего любое произвольное число аргументов командной строки, в том числе превышающее десять (рис. 4.5).

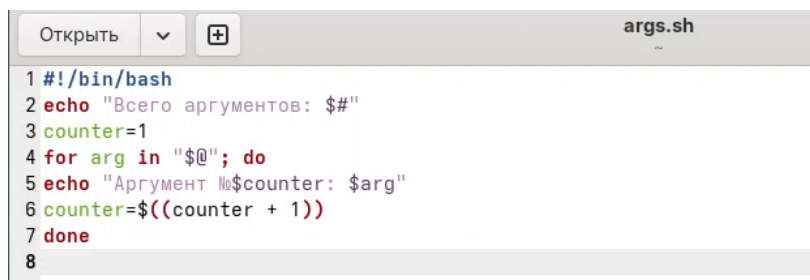


Рисунок 4.5: Скрипт для args.sh

Запустила с разными аргументами (рис. 4.6).

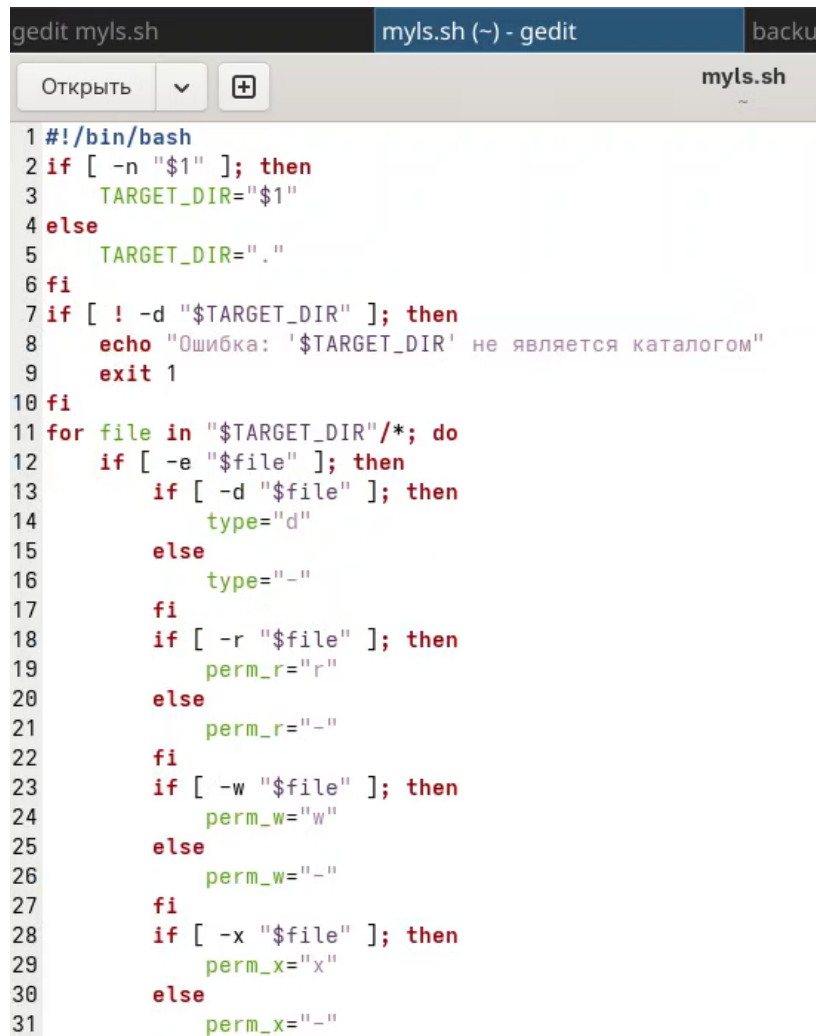
```

saterenteevskaya@saterenteevskaya:~$ chmod +x args.sh
saterenteevskaya@saterenteevskaya:~$ ./args.sh
Всего аргументов: 0
saterenteevskaya@saterenteevskaya:~$ ./args.sh 1 2 3 4 5 6 7 8 9 10 11
Всего аргументов: 11
Аргумент №1: 1
Аргумент №2: 2
Аргумент №3: 3
Аргумент №4: 4
Аргумент №5: 5
Аргумент №6: 6
Аргумент №7: 7
Аргумент №8: 8
Аргумент №9: 9
Аргумент №10: 10
Аргумент №11: 11
saterenteevskaya@saterenteevskaya:~$ ./args.sh a s f g h y r f v
Всего аргументов: 9
Аргумент №1: a
Аргумент №2: s
Аргумент №3: f
Аргумент №4: g
Аргумент №5: h
Аргумент №6: y
Аргумент №7: r
Аргумент №8: f
Аргумент №9: v

```

Рисунок 4.6: Запуск args.sh

3. Я написала командный файл — аналог команды ls (без использования самой этой команды и команды dir). Он выдает информацию о нужном каталоге и выводит информацию о возможностях доступа к файлам этого каталога (рис. 4.7), рис. 4.8)



```
1 #!/bin/bash
2 if [ -n "$1" ]; then
3     TARGET_DIR="$1"
4 else
5     TARGET_DIR="."
6 fi
7 if [ ! -d "$TARGET_DIR" ]; then
8     echo "Ошибка: '$TARGET_DIR' не является каталогом"
9     exit 1
10 fi
11 for file in "$TARGET_DIR"/*; do
12     if [ -e "$file" ]; then
13         if [ -d "$file" ]; then
14             type="d"
15         else
16             type="-"
17         fi
18         if [ -r "$file" ]; then
19             perm_r="r"
20         else
21             perm_r="-"
22         fi
23         if [ -w "$file" ]; then
24             perm_w="w"
25         else
26             perm_w="-"
27         fi
28         if [ -x "$file" ]; then
29             perm_x="x"
30         else
31             perm_x="-"
32         fi
33         echo "$type$perm_r$perm_w$perm_x $(basename "$file")"
34     fi
35 done
```

Рисунок 4.7: Скрипт для myls.sh

```
28     if [ -x "$file" ]; then
29         perm_x="x"
30     else
31         perm_x="-"
32     fi
33     echo "$type$perm_r$perm_w$perm_x $(basename "$file")"
34 fi
35 done
```

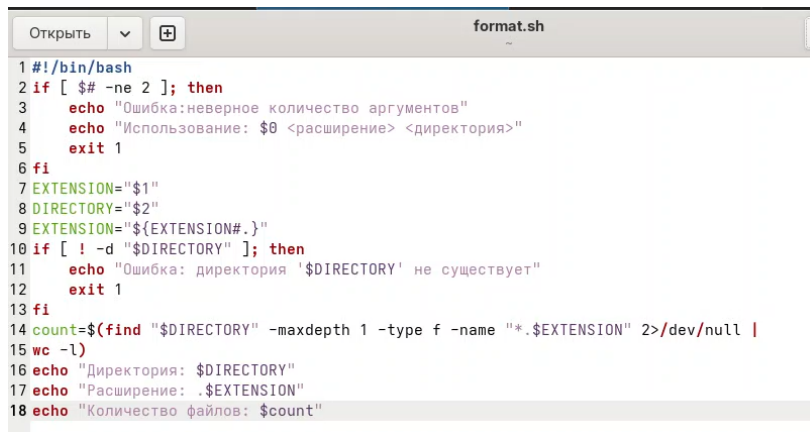
Рисунок 4.8: Скрипт для myls.sh

Запустила (рис. 4.9).

```
saterenteevskaya@saterenteevskaya:~$ chmod +x myls.sh
saterenteevskaya@saterenteevskaya:~$ ./mysls.sh
-rw- archiv.zip
-rwx args.sh
drwx backup
-rwx backup_self.sh
drwx bin
-rw- COURSE
drwx Documents
drwx Downloads
-rw- file1.txt
-rw- file2.txt
-rw- file3.txt
-rw- file4.txt
-rwx format.sh
drwx git-extended
dr-x gitflow
-rw- lab05_Терентеевская_архив.zip
-rw- lab11.sh
-rw- lab11.sh~
-rw- LICENSE
-rwx myls.sh
-rw- PkgTTC-Iosevka-34.2.1.zip
-rw- sway.log
drwx work
drwx Видео
drwx Документы
drwx Загрузки
drwx Изображения
drwx Музыка
drwx Общедоступные
drwx Рабочий стол
```

Рисунок 4.9: Запуск myls.sh

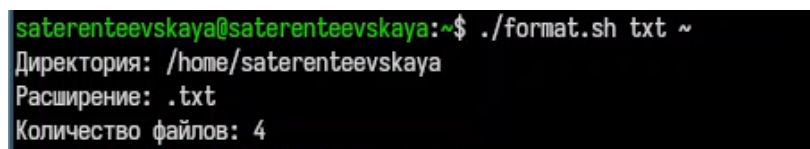
4. Я написала командный файл, который получает в качестве аргумента командной строки формат файла (.txt, .doc, .jpg, .pdf и т.д.) и вычисляет количество таких файлов в указанной директории. Путь к директории также передаётся в виде аргумента командной строки (рис. 4.10).

A screenshot of a text editor window titled "format.sh". The window has a menu bar with "Открыть", a dropdown arrow, and a "+" icon. The script content is as follows:

```
1 #!/bin/bash
2 if [ $# -ne 2 ]; then
3     echo "Ошибка: неверное количество аргументов"
4     echo "Использование: $0 <расширение> <директория>"
5     exit 1
6 fi
7 EXTENSION="$1"
8 DIRECTORY="$2"
9 EXTENSION="${EXTENSION#}"
10 if [ ! -d "$DIRECTORY" ]; then
11     echo "Ошибка: директория '$DIRECTORY' не существует"
12     exit 1
13 fi
14 count=$(find "$DIRECTORY" -maxdepth 1 -type f -name ".*$EXTENSION" 2>/dev/null |
15 wc -l)
16 echo "Директория: $DIRECTORY"
17 echo "Расширение: .$EXTENSION"
18 echo "Количество файлов: $count"
```

Рисунок 4.10: Скрипт для format.sh

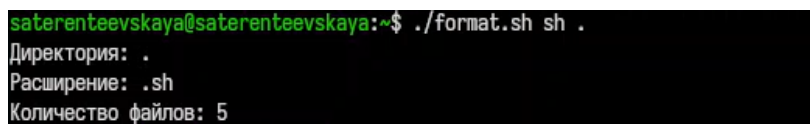
Запустила и проверила с txt (рис. 4.11).

A terminal window showing the execution of the script. The prompt is "saterenteevskaya@saterenteevskaya:~\$". The command entered is "./format.sh txt ~". The output is:

```
Директория: /home/saterenteevskaya
Расширение: .txt
Количество файлов: 4
```

Рисунок 4.11: Количество txt-файлов

и sh (рис. 4.12).

A terminal window showing the execution of the script. The prompt is "saterenteevskaya@saterenteevskaya:~\$". The command entered is "./format.sh sh .". The output is:

```
Директория: .
Расширение: .sh
Количество файлов: 5
```

Рисунок 4.12: Количество sh-файлов

5 Выводы

В ходе данной лабораторной работы я изучила основы программирования в оболочке ОС UNIX/Linux и научилась писать небольшие командные файлы.

6 Список литературы

ТУИС

Методичка к лабораторной работе №12